UNIVERSITY of HOUSTON
DEPARTMENT OF COMPUTER SCIENCE

Learning to Detect and Classify Malicious Executables in the Wild by J.Z. Kolter and M.A. Maloof

Rakesh Verma

Outline

Introduction

Selected previous work

Data Collection

Experimental Design

Experimental Results

Conclusion

Introduction

- Malware can
  - ✓ cause harm or subvert the system's intended function
- Malware can be classified into many categories. Three of them (considered by the authors) are:
  - ✓ viruses, worms, and Trojan horses.
- Authors use machine learning and data mining techniques to
  - ✓ Detect and classify malicious executables

Three main contributions

- Detect and classify malicious executables
  - ✓ Use text classification ideas with machine learning
- Present empirical results
  - ✓ from an extensive study of learning methods for detecting and classifying malicious executables
- Show that the methods achieve high detection rates
  - ✓ even on new malicious executables

Several learning methods

- Implemented in the Wakaito Environment for Knowledge Acquisition (WEKA)
  - ✓ Nearest neighbors (Ibk)
  - ✓ Naive Bayes
  - ✓ Support vector machine (SVM)
  - ✓ Decision trees (J48)
- Used the AdaBoost.M1 algorithm
  - ✓ Boost SVMs, J48, naive Bayes
  - ✓ Boosting nearest neighbors was too expensive

Selected Previous Work

- Schultz et al. used methods such as naïve Bayes for malware
  - ✓ Three feature extraction methods
  - ✓ Binary profiling: list of DLLs, function calls from DLLs, and number of distinct system calls from each DLL
  - ✓ String sequences: UNIX strings command to extract the printable strings in an object or binary file
  - ✓ Hex Dumps: similar to UNIX octal dump (od –x) command. Prints the contents of an executable file as a hexadecimal sequence
  - ✓ Each method is paired with a single learning algorithm.
  - ✓ Five-fold cross validation

Data Collection

- Gathered data in early 2003
  - ✓ Benign executables 1971
    - from Windows 2000 and XP operating systems
    - SourceForge
    - download.com
  - ✓ Malicious executables 1651
    - from Web site VX Heavens
    - MITRE Corporation, the sponsors of this project
- After experiments obtained 291 new malicious executables
  - ✓ from VX Heavens

Data Collection

- Hexdump utility used to convert data into hexadecimal codes in ASCII format
- Converted into 4-grams (why?) by combining each four-byte sequence into a single term.
  - ✓ Example: ab 12 bc 34 de 56 becomes ab12bc34, 12bc34de, and bc34de56
- Selected the top 500 4-grams from the training data

Experimental Design

- Evaluated the methods using
  - ✓ Ten-fold cross-validation
- Conducted ROC analysis for each method
- Three experimental studies:

1. Pilot study to determine:

- size of words and n-grams
- the number of n-grams relevant for prediction

2. Applied all of the classification methods to
   - a small collection of executables
3. Applied the methodology to
   - a larger collection of executables

Pilot Studies

- Sequential pilot studies to determine three parameters
  - ✓ The number of n-grams
  - ✓ The n for n-grams
  - ✓ The size of words
- Extracted bytes from
  - ✓ 476 malicious executables, 561 benign executables
  - ✓ produced n-grams, for n = 4
- Selected the best 10, 20, . . . , 100, 200, . . . , 1000, 2000, . . . , 10000 n-grams
  - ✓ Selecting 500 n-grams gave the best results

Pilot Studies

- Fixed the number of n-grams
  - ✓ at 500
  - ✓ varied n, the n-gram size
- Evaluated the same methods for n = 1, 2, ..., 10
  - ✓ n = 4 gave the best results
- Varied the size of the words (one byte, two bytes, etc.)
  - ✓ Single bytes gave better results

Feature Selection Details

- Formed training examples
  - ✓ Used the n-grams extracted from the executables
  - ✓ Each n-gram as a Boolean attribute
- Selected the most relevant attributes by
  - ✓ Computed the information gain (IG) for each:

$$IG(j) = \sum_{v_j \in \{0,1\}} \sum_{C_i} P(v_j, C_i) \log \frac{P(v_j, C_i)}{P(v_j)P(C_i)}$$

  - ✓ Ci denotes ith class, vj the value of the jth attribute

Small Collection Experiment

- Executables produced 68,744,909 distinct n-grams

- Areas under these curves (AUC) with 95% confidence intervals
  - ✓ Boosted methods performed well
  - ✓ Naive Bayes did not perform as well

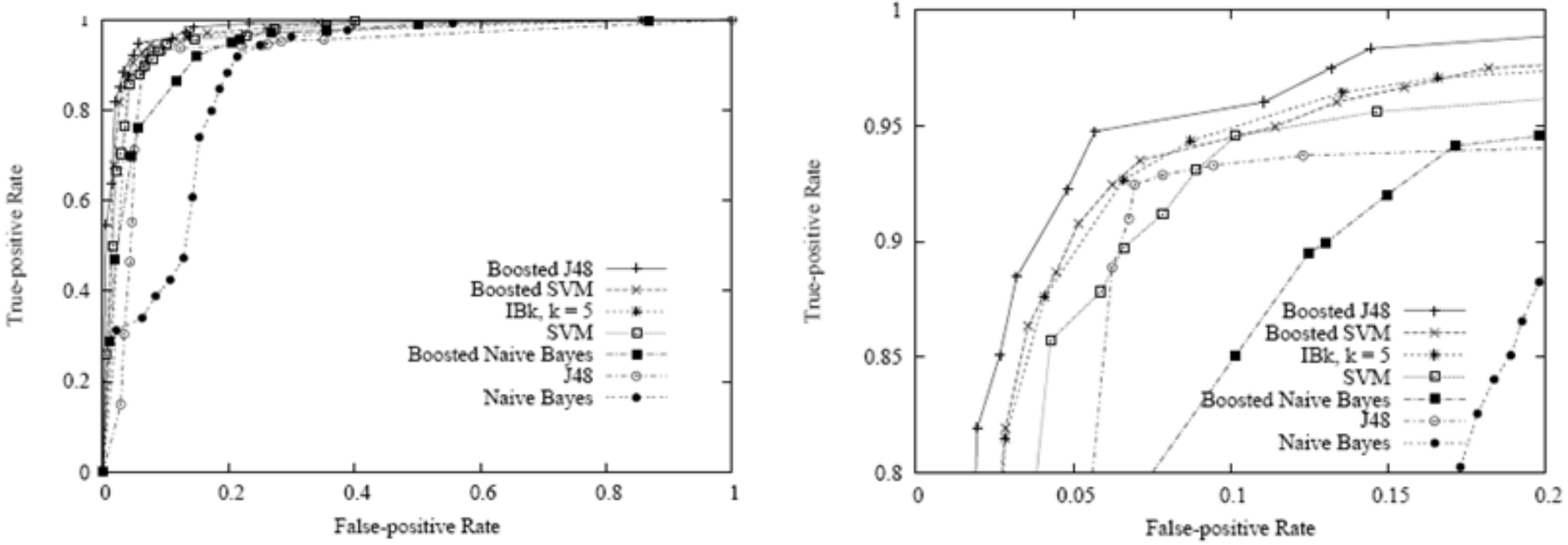

Figure: ROC curves for detecting malicious executables in the small collection.
Left: The entire ROC graph. Right: A magnification

Small Collection Experiment (cont.)

Method	AUC
Boosted J48	0.9836±0.0095
Boosted SVM	0.9744±0.0118
IBk, $k = 5$	0.9695±0.0129
SVM	0.9671±0.0133
Boosted Naive Bayes	0.9461±0.0170
J48	0.9235±0.0204
Naive Bayes	0.8850±0.0247

Table 2: Results for detecting malicious executables in the small collection. Measures are area under the ROC curve (AUC) with a 95% confidence interval.

Larger Collection Experiment

- Consisted of
  - ✓ 1971 benign executables
  - ✓ 1651 malicious executables
- over 255 million distinct 4-grams
- The areas under these curves with 95% confidence intervals
  - ✓ Boosted J48 outperformed all other methods

Larger Collection Experiment

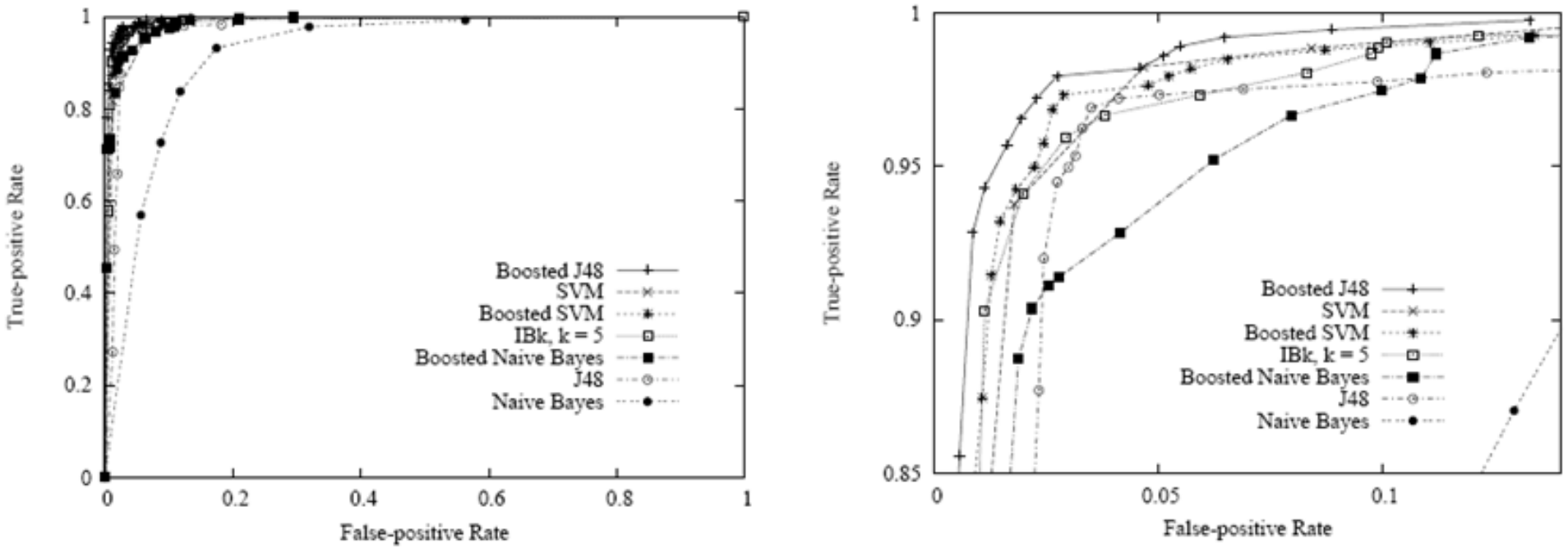

Figure: ROC curves for detecting malicious executables in the larger collection.
Left: The entire ROC graph, Right: A magnification

Larger Collection Experiment (cont.)

Method	AUC
Boosted J48	0.9958±0.0024
SVM	0.9925±0.0033
Boosted SVM	0.9903±0.0038
IBk, $k = 5$	0.9899±0.0038
Boosted Naive Bayes	0.9887±0.0042
J48	0.9712±0.0067
Naive Bayes	0.9366±0.0099

Table 3: Results for detecting malicious executables in the larger collection. Measures are area under the ROC curve (AUC) with a 95% confidence interval.

Classifying Executables by Payload Function

- Classify malicious executables based on
  - ✓ function of their payload
- Results for 3 functional categories
  1. opened a backdoor
  2. mass-mailed
  3. executable virus
- Reduce the effort to characterize previously undiscovered malicious executables
- One-versus-all classification
- Results not as good (refer to paper)

Evaluating Real-world, Online Performance

- Compare the actual detection rates
  - ✓ On 291 new malicious (no training on these)
- Selected three desired false-positive rates
  - ✓ 0.01, 0.05, 0.1
- Detected about 98% of the new malicious executables
  - ✓ Boosted J48
  - ✓ False-positive rate of 0.05

Evaluating Real-world, Online Performance

Method	Desired False-positive Rate					
	0.01		0.05		0.1	
	P	A	P	A	P	A
Boosted J48	0.94	0.86	0.99	0.98	1.00	1.00
SVM	0.82	0.41	0.98	0.90	0.99	0.93
Boosted SVM	0.86	0.56	0.98	0.89	0.99	0.92
IBk, $k = 5$	0.90	0.67	0.99	0.81	1.00	0.99
Boosted Naive Bayes	0.79	0.55	0.94	0.93	0.98	0.98
J48	0.20	0.34	0.97	0.94	0.98	0.95
Naive Bayes	0.48	0.28	0.57	0.72	0.81	0.83

Table 5: Results of a real-world, online evaluation. Predicted (P) versus actual (A) detect rates for three desired false-positive rates on 291 new, previously unseen malicious executables. Predicted detect rates are from Figure 2 and the experiment described in Section 5.4.

Conclusion

- Detecting and classifying unknown malicious executables by
  - ✓ Machine learning, data mining, text classification
- Detecting malicious executables
  - ✓ Boosted J48 produced the best detector with an area under the ROC curve of 0.996
- Classify malicious executables based on payload function
  - ✓ Boosted J48 produced the best detectors with areas under the ROC curve around 0.9

References

- Learning to Detect and Classify Malicious Executables in the Wild, JZ Kolter and MA Maloof, JMLR 7 (2006) 2721-2744
- Some slides adapted from: mmnet.iis.sinica.edu.tw/botnet/file/20100719/20100719_2.ppt